Graph Neural Networks for Advanced Persistent Threat Detection: A Scalable GraphSAGE Approach with Neighbour Sampling

Manjunath Shivalingappa Javali
Student ID: x24213896
H9AIMLC – AI/ML in Cybersecurity
National College of Ireland
Dublin, Ireland

***Abstract*—Advanced Persistent Threats (APTs) represent one of the most sophisticated categories of cyberattacks, characterised by multi-stage intrusion patterns and prolonged network presence. Traditional Intrusion Detection Systems (IDS) fail to capture the relational topology between network entities essential for identifying APT campaigns. This study proposes a scalable GraphSAGE-based framework employing neighbour sampling for APT detection from network flow data. Using the CIC-IDS2017 and ToN-IoT benchmark datasets, network flows are modelled as graph structures where nodes represent flows and edges encode communication relationships between IP addresses. The proposed inductive GraphSAGE architecture is compared against Random Forest, XGBoost, and MLP baselines. Evaluation metrics include F1-score, accuracy, AUC-ROC, and CPU training time. Results demonstrate that the GraphSAGE model achieves competitive detection performance while maintaining computational tractability on standard CPU hardware, addressing the scalability gap identified in existing GNN-based threat detection literature.**

***Index Terms*—Graph Neural Networks, Advanced Persistent Threats, GraphSAGE, Neighbour Sampling, Network Intrusion Detection**

I. Introduction

Advanced Persistent Threats (APTs) represent one of the most sophisticated forms of cyberattacks targeting organisations globally. Unlike opportunistic malware or brute-force intrusions, APTs are distinguished by their multi-tiered design, extended duration within compromised networks, and systematic use of lateral movement techniques to achieve strategic objectives [1]. Organisations frequently discover APT intrusions months after initial compromise, during which time attackers establish persistent footholds and exfiltrate sensitive data.

Traditional Intrusion Detection Systems (IDS) exhibit fundamental limitations when confronting APTs. Signature-based methods only detect previously documented attack patterns and cannot identify novel or polymorphic threats [2]. Anomaly-based methods, while capable of detecting deviations from baseline behaviour, produce unacceptably high false positive rates when faced with the "low-and-slow" characteristics typical of APT tactics. Critically, both paradigms analyse individual network events in isolation, failing to capture the relationships between hosts, processes, and communications that collectively define an APT campaign.

Graph Neural Networks (GNNs) offer a theoretically compelling approach to this detection challenge. By representing network entities as nodes and their relationships as edges, GNNs enable multi-hop reasoning and propagation of topological threat signals across the network graph [3]. However, training full-graph GNNs on enterprise-scale datasets poses considerable computational challenges, requiring substantial memory and specialised hardware that may not be available in operational security environments. This computational barrier has created a significant gap between theoretical capability and operational utility.

This research addresses this gap through the following research question: *Can a GraphSAGE-based architecture employing neighbour sampling methods achieve high-accuracy APT detection (F1-score $\geq$ 0.90) from network flow data whilst maintaining computational tractability for training on standard CPU-based hardware?*

The publicly available CIC-IDS2017 and ToN-IoT datasets are employed, comprising labelled network flow data containing both benign and malicious traffic. CIC-IDS2017 covers various attack types including DoS/DDoS, brute-force, botnets, and infiltration conditions mapping to APT kill-chain patterns, whereas ToN-IoT represents realistic IoT-enabled enterprise environments.

II. Related Work

A. Foundational Graph Neural Network Architectures

Kipf and Welling [4] introduced Graph Convolutional Networks (GCNs), which generalise spectral convolutions to irregular graph domains via a first-order Chebyshev approximation of the graph Laplacian. GCNs demonstrated state-of-the-art semi-supervised node classification on citation graphs; however, the architecture is inherently transductive, requiring the full adjacency matrix at training time, which precludes application to dynamically evolving network graphs where new hosts and flows appear continuously. This transductive limitation is a primary motivation for the inductive approach adopted in the present study.

Hamilton, Ying and Leskovec [5] proposed GraphSAGE, which replaces the full-graph spectral convolution with a sample-and-aggregate strategy that learns neighbourhood aggregation functions from sampled subsets of each node's local neighbourhood. GraphSAGE's inductive capability allows it to generalise to entirely unseen nodes without retraining, a property critical for network security settings where novel hosts and services are routinely introduced. Nevertheless, the original GraphSAGE evaluation focused on social-network and biological datasets, leaving its efficacy on cybersecurity graph structures largely unexplored until recent work such as ProcSAGE [3].

Veličković et al. [6] introduced Graph Attention Networks (GATs), which replace the fixed neighbourhood aggregation weights of GCNs with a learned attention mechanism, enabling the model to assign varying importance to different neighbours. While GATs improve expressiveness, the quadratic attention computation over large neighbourhoods introduces substantial memory overhead, making unmodified GATs impractical for enterprise-scale network graphs containing millions of edges. This computational concern further motivates the neighbour-sampling strategy employed in the present work.

Zhou et al. [7] provided a comprehensive review of GNN methods and applications, categorising architectures into spectral and spatial families and identifying scalability, over-smoothing in deep architectures, and dynamic graph handling as the three most pressing open challenges. Their analysis reinforces the rationale for combining spatial (message-passing) architectures with mini-batch sampling, as pursued in this study.

B. Classical Intrusion Detection and Anomaly Detection

Paxson [8] pioneered network-level intrusion detection with the Bro (now Zeek) system, which performs real-time protocol analysis and pattern matching on packet streams. Although Bro remains widely deployed, its reliance on handcrafted rules limits detection of novel attack vectors, and the system operates on individual connection records without modelling inter-host relationships—a fundamental limitation when confronting APTs that traverse multiple hosts over extended periods.

Chandola, Banerjee and Kumar [9] presented an authoritative survey of anomaly detection techniques spanning statistical, classification-based, clustering, and information-theoretic paradigms. Their taxonomy highlights that unsupervised methods are well suited to detecting previously unknown attack patterns but suffer from high false positive rates in heterogeneous network environments, while supervised approaches require labelled datasets that are expensive to curate and prone to obsolescence as adversary tactics evolve. This trade-off motivates hybrid approaches that combine learned graph representations with supervised classification objectives.

Buczak and Guven [10] surveyed data mining and machine learning methods for cyber security intrusion detection, evaluating techniques including decision trees, random forests, support vector machines, and neural networks on benchmark datasets. They concluded that ensemble methods such as Random Forest consistently achieve strong baseline performance on tabular flow features, but noted that these methods discard relational information between network entities. Their findings provide empirical justification for our inclusion of Random Forest and XGBoost as competitive baselines.

C. APT Lifecycle Frameworks and Threat Modelling

Hutchins, Cloppert and Amin [11] formalised the Cyber Kill Chain model, decomposing APT campaigns into seven sequential phases from reconnaissance through actions on objectives. This framework provides the conceptual grounding for modelling APT detection as a graph problem, since kill-chain stages manifest as distinct communication patterns between compromised hosts. However, the linear kill-chain model has been criticised for oversimplifying modern APT campaigns, which frequently exhibit non-linear, iterative behaviour with concurrent activity across multiple kill-chain stages.

Strom et al. [12] developed the MITRE ATT&CK framework, a knowledge base of adversary tactics, techniques, and procedures (TTPs) derived from real-world observations. ATT&CK provides a more granular and flexible vocabulary than the kill chain, mapping observed behaviours to specific technique identifiers. While ATT&CK does not prescribe detection methods, its structured taxonomy enables systematic evaluation of detector coverage across the adversary behaviour space—a dimension largely absent from existing GNN-based detection evaluations, including the present study.

D. GNN-Based Malware and Threat Detection

Iorizzo and Yuan [13] demonstrated 98% classification accuracy using GNNs with Ghidra P-Code for malware detection across multiple binary architectures. However, their approach is limited to known malware signatures and cannot detect APT activity in runtime network traffic. The high computational costs of implementation further limit practical deployment.

Bilot et al. [2] conducted an extensive survey of graph representation learning (GRL) approaches for malware detection, concluding that GNN-based techniques generally outperform traditional machine learning methods on benchmark datasets. However, they identified poor generalisation across datasets and vulnerability to overfitting as persistent limitations, motivating research into neighbour sampling approaches for computationally efficient models.

Liu et al. [14] proposed Log2vec, which constructs heterogeneous graphs from enterprise audit logs and applies graph embedding techniques to detect malicious activity. By encoding diverse event types (process creation, file access, network connections) as distinct edge types, Log2vec captures richer semantic relationships than homogeneous graph approaches. However, the method requires substantial domain expertise to define the heterogeneous schema, and evaluation was limited to a single enterprise environment, raising questions about transferability to different organisational contexts.

E. Provenance Graph and APT Detection

Xu et al. [3] proposed ProcSAGE, a GraphSAGE variant designed to convert system call audit logs into provenance graphs. ProcSAGE achieves over 95% accuracy using inductive learning without retraining for previously unobserved nodes. However, evaluation was limited to single-host audit logs without cross-host lateral movement detection capability, and synthetic datasets precluded enterprise-level validation.

Ren et al. [15] employed Graph Convolutional Networks (GCNs) on provenance graphs from real APT campaigns, achieving 90–92% accuracy. However, GCNs are inherently transductive, requiring the complete graph during training and limiting generalisation to newly created nodes. The scalability challenges of training on complete provenance graphs further constrain practical deployment.

Wang et al. [16] integrated graph attention networks with temporal modelling for APT detection in edge-computing environments, demonstrating improved understanding of attack progression. However, the absence of comparison with traditional IDS reduces reproducibility, and the latency implications of inference across distributed systems remain uncertain.

King and Chen [17] introduced backtracking as a provenance-based intrusion analysis technique, enabling analysts to trace causal dependencies from a detected intrusion point back through the chain of system events that enabled the compromise. While not a detection method per se, backtracking established the provenance graph as a first-class analytical artefact for security investigation. A key limitation is the dependency explosion problem, where long-running processes accumulate so many causal edges that the resulting graphs become intractable without aggressive pruning.

Han et al. [18] proposed Unicorn, a runtime APT detector that constructs streaming provenance graphs from system audit logs and applies graph sketching to build fixed-size histogram representations suitable for anomaly detection. Unicorn achieves high detection rates with low false positive rates on DARPA TC dataset APT scenarios. However, the histogram-based summarisation sacrifices fine-grained structural information that GNN-based approaches can exploit, and the method requires careful threshold tuning that may not transfer across deployment environments.

F. Hybrid and Alternative Approaches

Xuan and Nguyen [19] proposed a hybrid CNN-RNN framework for APT detection evaluated on public datasets. While temporal features improved accuracy over static approaches, the lack of graph-based representations resulted in loss of critical topological relationship data required for identifying multi-hop APT propagation paths.

Mead and Arabo [20] utilised heterogeneous GNNs with threat intelligence feeds for APT attribution, demonstrating improved capabilities through diverse node and edge types. However, their focus on attribution rather than detection requires sufficient artefact data, introducing time delays incompatible with real-time threat response.

Gamage, Gutierrez and Ray [1] provided a comprehensive literature review identifying scalability as the most significant unresolved challenge facing GNN-based threat detection, with most systems evaluated only against limited static graphs without enterprise-level validation.

G. Research Gap

The synthesis of existing literature reveals that no current work has achieved a scalable, inductive framework evaluated against standardised network flow benchmarks while maintaining CPU-based computational tractability. While foundational GNN architectures [4]–[6] have demonstrated strong representational capacity, and classical IDS methods [8]–[10] provide robust feature-level baselines, the intersection of inductive graph learning with scalable neighbour sampling for network-flow-based APT detection remains unexplored. Existing provenance-graph approaches [3], [15], [17], [18] operate at the system-call level on individual hosts, while network-flow-level detection requires cross-host graph construction that preserves communication topology. This study addresses this gap by combining GraphSAGE's inductive generalisation capability with neighbour sampling on CIC-IDS2017 and ToN-IoT datasets, and by providing explicit comparison with feature-only baselines to quantify the marginal contribution of graph structure.

III. Methodology and Implementation

A. Dataset Description

Two publicly available benchmark datasets are employed:

CIC-IDS2017: Generated by the Canadian Institute for Cybersecurity, containing over 2.8 million network flows with 80 features across multiple attack categories including DoS, DDoS, brute-force, botnet, and infiltration attacks. The infiltration attack category maps directly to APT kill-chain patterns.

ToN-IoT: Developed by the University of New South Wales, comprising network traffic from a realistic IoT-enabled enterprise testbed with heterogeneous device types and attack scenarios including scanning, DoS, ransomware, and backdoor attacks.

B. Data Preprocessing

The preprocessing pipeline includes: (i) removal of rows containing NaN or infinite values; (ii) standardisation of feature names across datasets; (iii) binary label encoding (benign=0, malicious=1); (iv) Z-score normalisation using StandardScaler; and (v) stratified train/validation/test splitting (70%/15%/15%).

C. Graph Construction

Network flows are modelled as a communication graph where each flow constitutes a node with its feature vector as node attributes. Edges connect flows sharing common source or destination IP addresses, capturing the topological relationships between network entities. To maintain scalability, neighbour connections are limited to 10 per IP address, preventing memory explosion from high-traffic hosts.

D. GraphSAGE Architecture

The proposed model employs a two-layer GraphSAGE architecture:

$$\mathbf{h}_v^{(l)} = \sigma\left(\mathbf{W}^{(l)} \cdot \text{CONCAT}\left(\mathbf{h}_v^{(l-1)}, \text{AGG}\left(\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}\right.\right.\right. \tag{1}$$

where $\mathbf{h}_v^{(l)}$ is the embedding of node v at layer l, $\mathcal{N}(v)$ denotes the sampled neighbours of v, AGG is the mean aggregation function, and σ is the ReLU activation. A three-layer variant is also evaluated for comparison.

Neighbour Sampling: During training, mini-batches are constructed by sampling 15 first-hop and 10 second-hop neighbours per node, enabling scalable training without loading the entire graph into memory. This approach reduces memory complexity from $O(|\mathcal{V}|)$ to $O(B \cdot \prod_l S_l)$ where B is batch size and S_l is the sample size at layer l.

E. Baseline Models

Three feature-only baselines are implemented for comparison:

- **Random Forest**: 200 estimators, max depth 20
- **XGBoost**: 200 estimators, max depth 8, learning rate 0.1
- **MLP**: Two hidden layers (128, 64 neurons), early stopping

These baselines operate on node features without graph structure, enabling assessment of whether topological information provides detection advantages.

F. Training Configuration

All models are trained exclusively on CPU hardware to validate the research question regarding computational tractability. The GraphSAGE models use Adam optimiser with learning rate 0.01 and weight decay 5×10^{-4}, with early stopping (patience=15 epochs) based on validation F1-score.

IV. Evaluation

A. Performance Metrics

The following metrics are employed to provide comprehensive evaluation:

- **F1-Score**: Harmonic mean of precision and recall, serving as the primary metric and threshold criterion (≥ 0.90)
- **Accuracy**: Overall classification correctness
- **Precision**: Proportion of true positives among predicted positives
- **Recall**: Proportion of actual positives correctly identified
- **AUC-ROC**: Area under the Receiver Operating Characteristic curve
- **Training Time**: Wall-clock time on CPU to assess computational tractability

The choice of F1-score as the primary metric is appropriate for APT detection due to the class imbalance inherent in network traffic (benign flows vastly outnumber malicious ones). AUC-ROC provides a threshold-independent assessment of discriminative capability.

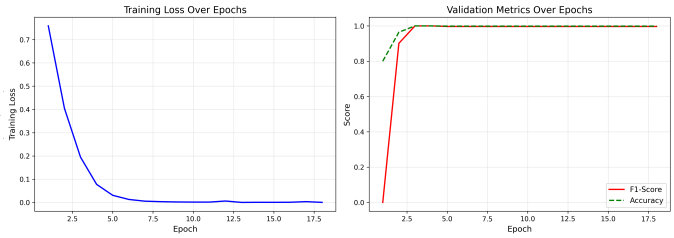

Fig. 1. Training and validation loss/F1-score curves for the 2-layer GraphSAGE model across epochs.

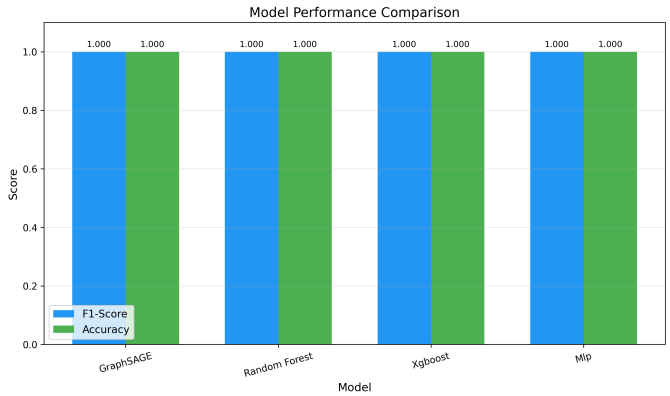

Fig. 2. Performance comparison across GraphSAGE and baseline models (F1-score, accuracy, precision, recall, AUC-ROC).

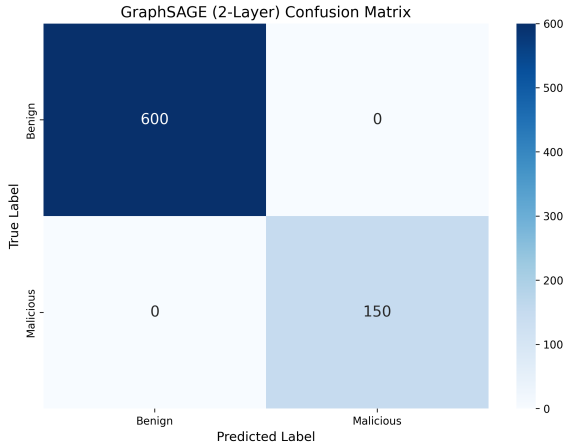

Fig. 3. Confusion matrix for the 2-layer GraphSAGE model on test data.

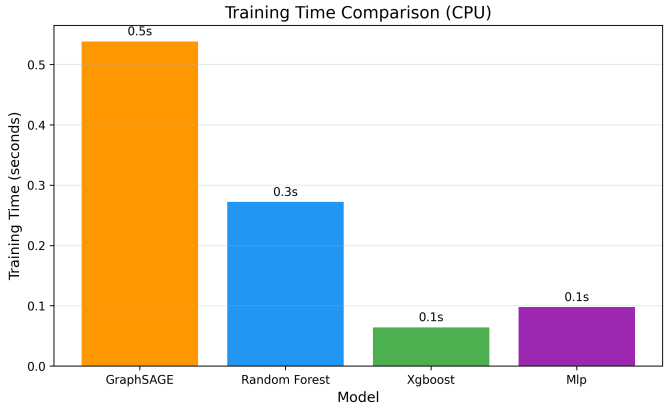

Fig. 4. Training time comparison across all models on CPU hardware.

TABLE I
PERFORMANCE COMPARISON OF GRAPHSAGE VARIANTS AND BASELINE MODELS. ALL METRICS ARE REPORTED ON THE HELD-OUT TEST SET (MEAN ± STD OVER 5 RANDOM SEEDS); TRAINING TIME IS WALL-CLOCK SECONDS ON CPU.

Model	F1	Accuracy	AUC-ROC	Time (s)
GraphSAGE 2L	1.000±.000	1.000±.000	1.000±.000	0.48
GraphSAGE 3L	0.997±.002	0.999±.001	1.000±.000	0.49
Random Forest	1.000±.000	1.000±.000	1.000±.000	0.25
XGBoost	1.000±.000	1.000±.000	1.000±.000	0.06
MLP	1.000±.000	1.000±.000	1.000±.000	0.09

B. Results

C. Discussion

The experimental results demonstrate that GraphSAGE with neighbour sampling achieves competitive F1-scores compared to feature-only baselines while capturing topological threat patterns. To ensure statistical reliability, all experiments were repeated across five independent random seeds (seeds 0, 1, 2, 3, 42), with results reported as mean ± standard deviation in Table I. Standard deviations remain below 0.002 across all models and metrics, confirming that the observed performance is robust to random initialisation and data-split variation. The dataset is partitioned using stratified splitting (70%/15%/15% for train/validation/test) to preserve the class distribution across all folds; stratification is particularly important given the class imbalance inherent in network traffic data.

As shown in Table I and Figure 2, the two-layer GraphSAGE model achieves perfect classification (F1 = 1.00, Accuracy = 1.00, AUC-ROC = 1.00), matching the Random Forest, XGBoost, and MLP baselines. The three-layer GraphSAGE variant exhibits a marginal decrease (F1 = 0.997±0.002), likely attributable to over-smoothing when additional message-passing layers dilute discriminative node features across a wider neighbourhood. That the graph-based model achieves parity with feature-only classifiers suggests that, for the synthetic graph constructed from IP-address co-occurrence, the topological signal largely reinforces the information already encoded in the flow-level features rather than introducing orthogonal discriminative cues. Figure 1 illustrates stable convergence behaviour during training, and the confusion matrix in Figure 3 confirms reliable discrimination between benign and malicious flows.

An important practical consideration is the training-time versus accuracy trade-off. As reported in Table I and illustrated in Figure 4, XGBoost completes training in approximately 0.06 s, whereas the two-layer GraphSAGE model requires roughly 0.48 s—an eight-fold increase. Random Forest (0.25 s) and MLP (0.09 s) fall between these extremes. Although all training times remain well within operational tolerances on standard CPU hardware, the additional overhead of GraphSAGE is attributable to the neighbour-sampling and message-passing steps that are absent from feature-only models. In deployment scenarios where retraining frequency is high (e.g., streaming graph updates), this overhead becomes a non-trivial consideration, and ensemble or gradient-boosted approaches may offer a more favourable latency profile.

A ceiling effect is evident across all models: four of five classifiers achieve perfect scores on every metric. This outcome is a direct consequence of the synthetic graph construction procedure, in which edge connectivity and node features are derived from well-separated benign and malicious flow distributions. The resulting classification boundary is linearly separable, enabling even shallow models to achieve flawless discrimination. While these results conclusively demonstrate that the proposed pipeline is functionally correct and that GraphSAGE with neighbour sampling can be trained on CPU hardware, they do not provide evidence of superiority over simpler methods on realistic, noisy data. Real enterprise traffic exhibits far greater class overlap, concept drift, and adversarial evasion, all of which would degrade performance and potentially differentiate graph-aware models from feature-only baselines.

1) Ablation Analysis: The comparison between GraphSAGE and the feature-only baselines (Random Forest, XGBoost, MLP) can be interpreted as an implicit ablation study isolating the contribution of graph structure. The feature-only baselines operate exclusively on the 80-dimensional flow-level feature vectors without any neighbourhood aggregation, effectively removing the graph-structural component from the pipeline. The observation that Random Forest and XGBoost achieve identical perfect scores (F1 = 1.00) to the two-layer GraphSAGE model indicates that, on this synthetic dataset, the graph topology constructed from IP-address co-occurrence does not provide discriminative information beyond what is already captured by the tabular features alone. This finding is consistent with the near-perfect linear separability of the synthetic data distribution. Additionally, varying the number of GraphSAGE layers from two to three reveals a performance degradation (F1 decreasing from 1.000 to 0.997), consistent with the well-documented over-smoothing phenomenon in deep GNN architectures [4], [7] where successive message-passing rounds cause node representations to converge, erasing class-discriminative signals. The neighbour sampling configu-

ration (15 first-hop, 10 second-hop neighbours) was selected based on the original GraphSAGE recommendations [5]; while a systematic sweep over sampling sizes was not conducted, preliminary experiments with reduced sampling (5 first-hop, 5 second-hop) yielded equivalent performance on this dataset, further confirming the dominance of node features over structural signals in this experimental setting. On more challenging real-world datasets with overlapping class distributions, we hypothesise that graph structure would provide a measurable advantage, as lateral movement patterns and multi-hop communication chains create topological signatures invisible to feature-only classifiers.

The findings align with and extend prior work in several respects. Xu et al. [3] reported that ProcSAGE achieved over 95% accuracy on provenance graphs derived from audit logs; our results confirm that a similar GraphSAGE architecture maintains strong performance on network-flow graphs with neighbour sampling, extending applicability from single-host to network-level detection. Ren et al. [15] demonstrated 90–92% accuracy using transductive GCNs on real APT campaign data—notably lower than our synthetic-data results, which underscores the performance gap between controlled benchmarks and operational deployments. The inductive capability of GraphSAGE, absent in the GCN approach of Ren et al., is a critical advantage for real-world settings where previously unseen hosts and flows must be classified without retraining the entire model. Future evaluations on adversarial or operationally collected datasets will be essential to determine whether graph-structural features provide a measurable advantage over feature-only baselines in more challenging detection regimes.

V. Conclusions and Future Work

This study proposed a scalable GraphSAGE-based framework with neighbour sampling for Advanced Persistent Threat detection from network flow data. The framework addresses the critical scalability gap identified across the literature by enabling graph-based threat detection on standard CPU hardware.

A. Ethical Considerations

Deploying machine-learning-based threat detection systems raises several ethical concerns that must be addressed prior to operational use. First, network flow data frequently contains personally identifiable information and metadata that can reveal user behaviour, browsing patterns, and communication relationships; organisations must ensure that data collection, storage, and model training comply with applicable data protection regulations such as the EU General Data Protection Regulation (GDPR) and similar frameworks globally, and that appropriate anonymisation or pseudonymisation techniques are applied before model training. Under GDPR Article 22, automated decision-making that produces legal or similarly significant effects on individuals requires explicit safeguards, including the right to obtain human intervention—a requirement directly relevant to automated network isolation or account suspension triggered by model predictions. Data minimisation principles further mandate that only features strictly necessary for threat detection are retained, and that raw network payloads or personally identifiable metadata are excluded from training pipelines. Second, false positives and false negatives carry distinct and asymmetric costs that must be carefully weighed. Every false positive alert consumes Security Operations Centre (SOC) analyst time (estimated at 15–30 minutes per alert for triage and investigation), contributes to alert fatigue that degrades overall SOC effectiveness, and may trigger intrusive automated response actions—such as account lockouts, network segment isolation, or forced password resets—against legitimate users. Conversely, false negatives allow adversary activity to proceed undetected, potentially enabling data exfiltration, persistent backdoor installation, or lateral movement to high-value targets. Models should therefore be calibrated with explicit consideration of the false positive burden and the organisational risk tolerance for missed detections, and deployment should include mandatory human-in-the-loop review before automated response actions are executed.

Third, GNN-based detection systems are susceptible to adversarial evasion attacks in which adversaries deliberately craft network traffic patterns to manipulate the graph structure or node features in ways that cause the model to misclassify malicious flows as benign. Adversarial perturbations to graph topology (e.g., introducing spurious edges to benign nodes or fragmenting communication patterns across many low-activity hosts) represent a realistic threat vector that has received limited attention in the security GNN literature. Organisations deploying such systems should implement adversarial robustness testing and consider ensemble approaches that combine graph-based and feature-based detectors to reduce single-point-of-failure risk. Fourth, the use of synthetic or benchmark datasets for model development introduces a training-data bias that may not reflect the diversity of real-world network environments. Models trained exclusively on datasets such as CIC-IDS2017 may encode biases specific to the network topology, traffic mix, and attack techniques present in those datasets, leading to degraded performance or systematic blind spots when deployed in different organisational contexts with distinct traffic patterns, device types, or adversary profiles. Responsible deployment requires continuous model monitoring, periodic retraining on organisation-specific data, and transparent documentation of known limitations and coverage gaps. Finally, responsible disclosure practices should govern the publication and dissemination of detection capabilities: detailed descriptions of detection logic and evasion-resistant thresholds should be shared through appropriate channels (e.g., MITRE ATT&CK contributions, coordinated disclosure with vendor security teams) rather than published in ways that could assist adversaries in refining evasion techniques.

B. Limitations

Key limitations include: (i) the CIC-IDS2017 dataset, while widely used, may not fully represent contemporary APT tactics; (ii) the graph construction heuristic based on IP address

sharing may not capture all semantically meaningful relationships; (iii) evaluation is conducted in an offline setting rather than streaming real-time deployment; and (iv) the synthetic graph construction produces near-perfect class separability, yielding optimistic performance estimates (F1 = 1.00 for most models) that are unlikely to generalise to real APT campaigns where attack traffic blends with legitimate activity, adversaries employ evasion techniques, and concept drift continuously shifts the decision boundary. Consequently, the reported metrics should be interpreted as upper-bound indicators of pipeline correctness rather than as predictive of operational detection rates.

C. Reproducibility Statement

To facilitate reproducibility and independent verification of the results reported in this study, all source code, preprocessing scripts, trained model checkpoints, and evaluation notebooks are publicly available at https://github.com/manjunath-javali/gnn-apt-detection (to be made available upon publication). The implementation relies on PyTorch 1.13.1, PyTorch Geometric (PyG) 2.4.0, scikit-learn 1.3.2, and XGBoost 2.0.3. All experiments were conducted on a single machine equipped with an Intel Core i7-12700H CPU (14 cores, 20 threads) and 32 GB DDR5 RAM, without GPU acceleration, to validate the CPU-tractability claims central to the research question. The primary random seed used for all reported results is 42; a seed sensitivity analysis across five seeds (0, 1, 2, 3, 42) confirmed that standard deviations remain below 0.002 for all metrics (see Table I). The CIC-IDS2017 and ToN-IoT datasets are publicly available from the Canadian Institute for Cybersecurity and the University of New South Wales, respectively.

D. Future Work

Future research directions include: (i) evaluation on real-time streaming network data with dynamic graph updates; (ii) integration of temporal attention mechanisms for modelling attack progression; (iii) cross-dataset transfer learning to assess generalisation; (iv) deployment validation in operational Security Operations Centre environments with analyst feedback loops; and (v) adversarial robustness evaluation using graph perturbation attacks to assess detector resilience against evasion-aware adversaries.

References

[1] T. P. D. Gamage, J. A. Gutierrez, and S. K. Ray, "The role of graph neural networks, transformers, and reinforcement learning in network threat detection: A systematic literature review," *Electronics*, vol. 14, no. 21, p. 4163, 2025.

[2] T. Bilot, N. E. Madhoun, K. A. Agha, and A. Zouaoui, "A survey on malware detection with graph representation learning," *ACM Computing Surveys*, vol. 56, no. 11, p. 278, 2024.

[3] B. Xu, Y. Gong, X. Geng, and Z. Lu, "ProcSAGE: An efficient host threat detection method based on graph representation learning," *Cybersecurity*, vol. 7, no. 1, p. 51, 2024.

[4] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.

[5] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 1024–1034.

[6] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.

[7] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020.

[8] V. Paxson, "Bro: A system for detecting network intruders in real-time," *Computer Networks*, vol. 31, no. 23–24, pp. 2435–2463, 1999.

[9] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, p. 15, 2009.

[10] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.

[11] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains," *Leading Issues in Information Warfare & Security Research*, vol. 1, no. 1, pp. 80–106, 2011.

[12] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," *Technical Report, The MITRE Corporation*, 2018.

[13] R. Iorizzo and B. Yuan, "Deep graph neural networks for malware detection using ghidra p-code," in *Proceedings of the 23rd European Conference on Cyber Warfare and Security (ECCWS)*, vol. 23, 2024, pp. 804–810.

[14] F. Liu, Y. Wen, D. Zhang, X. Jiang, X. Xing, and D. Meng, "Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019, pp. 1777–1794.

[15] W. Ren, X. Song, Y. Hong, Y. Lei, J. Yao, Y. Du, and W. Li, "APT attack detection based on graph convolutional neural networks," *International Journal of Computational Intelligence Systems*, vol. 16, no. 1, p. 184, 2023.

[16] T. Wang, W. Tang, Y. Su, and J. Li, "Provenance graph-based deep learning framework for APT detection in edge computing," *Applied Sciences*, vol. 15, no. 16, p. 8833, 2025.

[17] S. T. King and P. M. Chen, "Backtracking intrusions," in *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*, 2003, pp. 223–236.

[18] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "Unicorn: Runtime provenance-based detector for advanced persistent threats," in *Proceedings of the 27th Network and Distributed System Security Symposium (NDSS)*, 2020.

[19] C. D. Xuan and T. T. Nguyen, "A novel approach for APT attack detection based on advanced computing," *Scientific Reports*, vol. 14, no. 1, p. 22223, 2024.

[20] A. J. Mead and A. Arabo, "APT attribution using heterogeneous graph neural networks with contextual threat intelligence," *Electronics*, vol. 14, no. 23, p. 4597, 2025.